



Fundamentals of Android Applications

UNIT-1

Topics

- ▶ OOPS
- ▶ Software Engineering
- ▶ Basics of Android and Android application

Why we use java

- ▶ Java is a popular and widely-used programming language for many reasons. Here are some key reasons why developers choose Java:
- ▶ **1. Platform Independence**
- ▶ **Write Once, Run Anywhere (WORA):** Java code is compiled into bytecode, which can be run on any device equipped with the Java Virtual Machine (JVM). This platform independence is one of Java's most significant advantages.
- ▶ **2. Object-Oriented Programming**
- ▶ **Modularity and Reusability:** Java's object-oriented nature encourages the use of modular code, which can be reused across different programs. This leads to better organized and more manageable code.



▶ 3. Rich Standard Library

- ▶ **Extensive APIs(Application Programming Interface)** : Java has a vast standard library that provides many useful APIs for tasks like I/O operations, networking, data structures, and much more, reducing the need to write code from scratch.

- ▶ **Work of API**-An API is a set of rules and protocols that allows different software applications to communicate with each other. It defines the methods and data structures that developers can use to interact with the functions and features provided by a particular software component or service.

▶ 4. Strong Memory Management

- ▶ **Automatic Garbage Collection:** Java handles memory allocation and deallocation automatically through its garbage collection mechanism, reducing the likelihood of memory leaks and other related issues.

▶ **5. Robust and Secure**

- ▶ **Strong Exception Handling:** Java provides a robust framework for exception handling, making it easier to write error-free code.

Security Features: Java offers comprehensive security features, including bytecode verification, sandboxing, and built-in security APIs.

▶ **6. Multi-threading Capability**

- ▶ **Concurrency Support:** Java has built-in support for multi-threading, allowing developers to write programs that can perform multiple tasks simultaneously, leading to more efficient and responsive applications.

- ▶ **. Community and Ecosystem**

- ▶ **Large Developer Community:** Java has a vast and active developer community, offering extensive documentation, tutorials, and third-party libraries and frameworks. **Enterprise Adoption:** Java is widely used in enterprise environments, making it a valuable skill for developers in the job market.

- ▶ **8. Development Tools**

- ▶ **Integrated Development Environments (IDEs):** There are several powerful IDEs available for Java development, such as Eclipse, IntelliJ IDEA, and NetBeans, which provide tools for code completion, debugging, and project management.

Environments

► Integrated Development Environments (IDEs):

- **Eclipse:** A popular, open-source IDE for Java development.
- **IntelliJ IDEA:** A feature-rich, commercial IDE with a free community edition.
- **NetBeans:** An open-source IDE supported by the Apache Software Foundation.
- **Visual Studio Code:** A lightweight, open-source editor that supports Java through extensions.

OOP

- ▶ Object-Oriented Programming (OOP) is a programming paradigm centered around the concept of "objects," which are instances of classes. OOP is designed to help organize and structure software in a more modular and reusable way. The four fundamental principles of OOP are encapsulation, inheritance, polymorphism, and abstraction. Here's a detailed explanation of each concept:

Class

▶ Class in Java

- ▶ **Definition:** A class in Java is a blueprint or template for creating objects. It defines a set of properties (fields) and methods (functions) that the objects created from the class will have. A class encapsulates data for the object and methods to manipulate that data.

▶ Components of a Class:

- **Fields (Variables):** Represent the state or attributes of the class.
- **Methods:** Define the behavior or functions that the objects of the class can perform.
- **Constructors:** Special methods used to initialize objects.

Functions

A code which gets executed to perform any task is called Function.

► Example.

$$\begin{array}{r} 87 \\ + 87 \\ \hline \end{array}$$

Function and Class

► Function →

CARPENTER

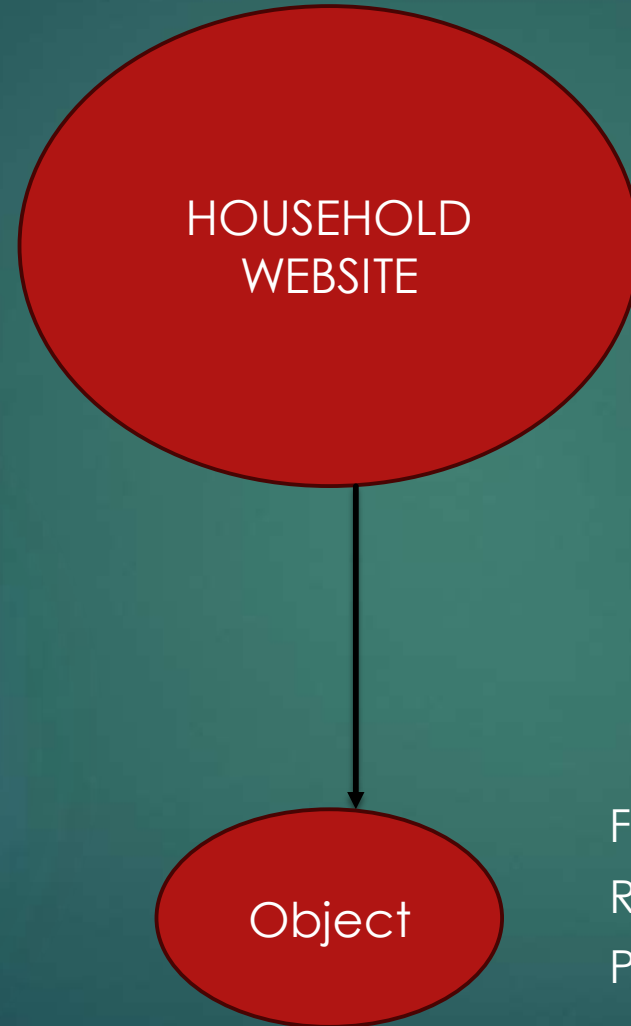
A code which gets executed to perform any task

► Class →

HOUSEHOLD
REPAIR

Group

Object



Form creation :Kind of Work
Repair Name
Payment

Objects

An object is an instance of a class. When a class is defined, no memory is allocated until an object of that class is created. Objects are the real-world entities that hold specific data and have specific behavior defined by the class.

Creating an Object:

- Use the new keyword followed by the class constructor.
- The constructor initializes the new object.

Example

► Package com.website

```
public class Main
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
    //Write your code here
```

```
    System.out.println("Hello World");
```

```
}
```

```
}
```

Let us understand .

- ▶ Com.company ; ----A package in Java is used to group which can be related to class.
- ▶ Public --- This means this method can be accessed from anywhere.
- ▶ Main --- It is the class.
- ▶ Static --- This helps the program without any object.
- ▶ Void ---- In this program the function does not provide any return so it is empty that's why we used void here.

Continue...

- `main` --- Functions
- `System.out` –Output
- `Println` ---This is the general action performed by the method. It takes data and writes it to the console (or another output stream). In: This most likely stands for "line" because the `println` method does two things:
 - Prints the provided data to the console.
 - Moves the cursor to the next line after printing.

Naming Conventions



PascalCaseConvention



camelCaseConvention

Pascal---Class

For class we use Pascal Convention.

Example

HelloWorld----Both starting letters will be in capital.

Main---First letter in capital.

camelCaseConvention--Function

- ▶ In camel case convention we use first word's first letter small and all other words first letter in capital to define a function
- ▶ Example
- ▶ helloWorld—Firstword first letter small.
- ▶ main

Encapsulation

- ▶ Encapsulation is a mechanism through which we can wrap the data members and members methods of class in a single unit called encapsulation.
- ▶ **Note**
- ▶ Declare the class variables as private.
- ▶ Declare the class methods as public.

Users > Nair Renjini > Desktop > Java > J B.java > A > setValue(int)

```
//Encapsulation is a method by which we wrap data
class A
{
    private int value;//data hiding //value=variable
    public void setValue(int x)//data abstraction
    {
        value=x;//value=90
    }
    public int getValue()
    {
        return ++value;
    }
}
class B
{
    Run | Debug
    public static void main(String[] args){
        A r=new A();//r is object here in which value is stored
        r.setValue(x:90);
        System.out.print(r.getValue());
    }
}
```

Output

PROBLEMS (4) OUTPUT DEBUG CONSOLE

```
PS C:\Users\Nair Renjini> ^C
```

```
PS C:\Users\Nair Renjini>
```

```
PS C:\Users\Nair Renjini> & 'C:\Pro  
-cp' 'C:\Users\Nair Renjini\AppData\  
91
```

```
PS C:\Users\Nair Renjini>
```

Inheritance

- ▶ Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a new class to inherit properties and behaviors (fields and methods) from an existing class. This promotes code reuse and establishes a natural hierarchical relationship between classes.

Key Concepts of Inheritance

1. Superclass (Parent Class)

- The class whose properties and methods are inherited by another class.
- Also known as the base class or parent class.

2. Subclass (Child Class)

- The class that inherits properties and methods from another class.
- Also known as the derived class or child class.

3. extends Keyword

- Used to declare a subclass. The extends keyword is followed by the name of the superclass.

Types of Inheritance

- ▶ Single / Simple Inheritance
- ▶ Multi- Level Inheritance
- ▶ Hierarchical Inheritance

Syntax

- ▶ Class A

- ▶ {

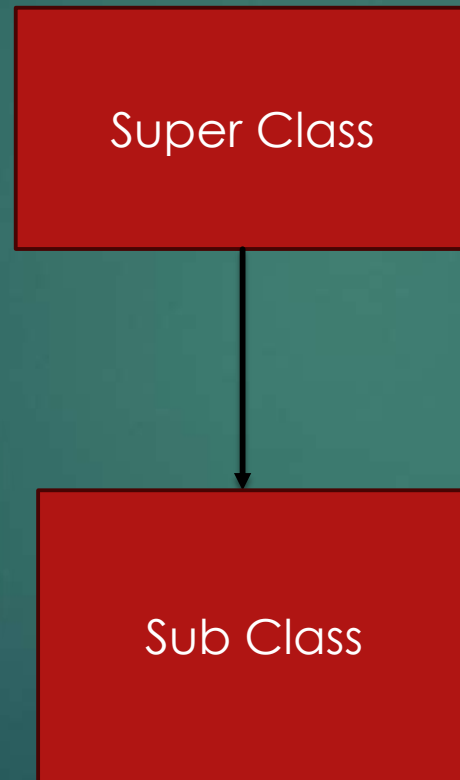
- ▶ }

- ▶ Class B extends A

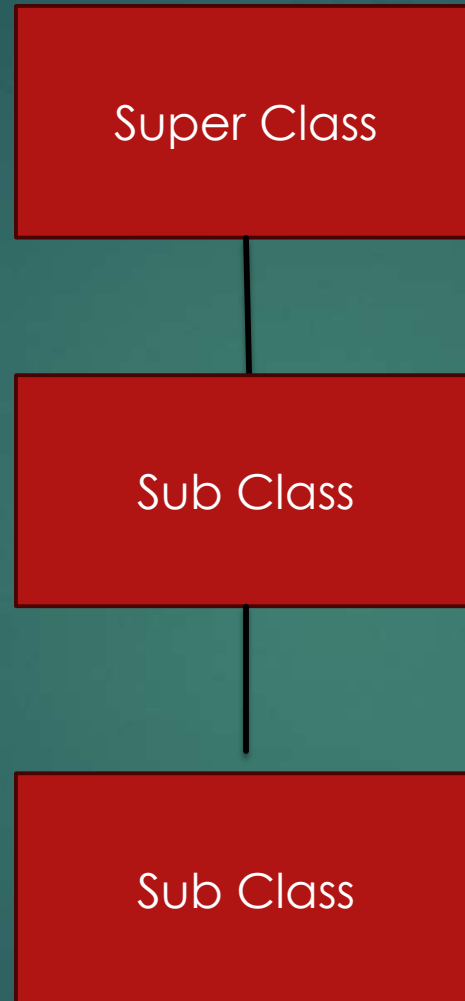
- ▶ {

- ▶ }

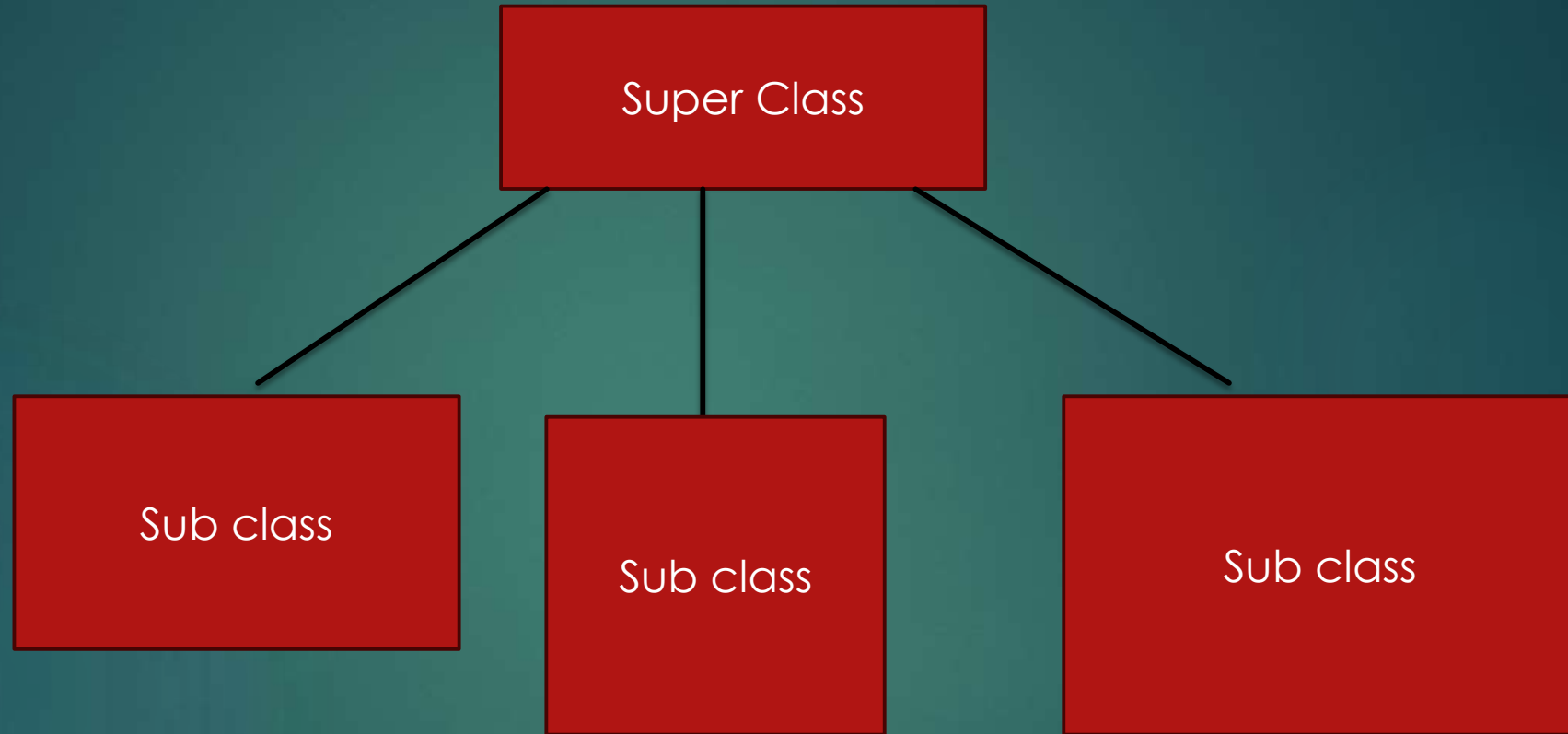
Single / Simple Inheritance



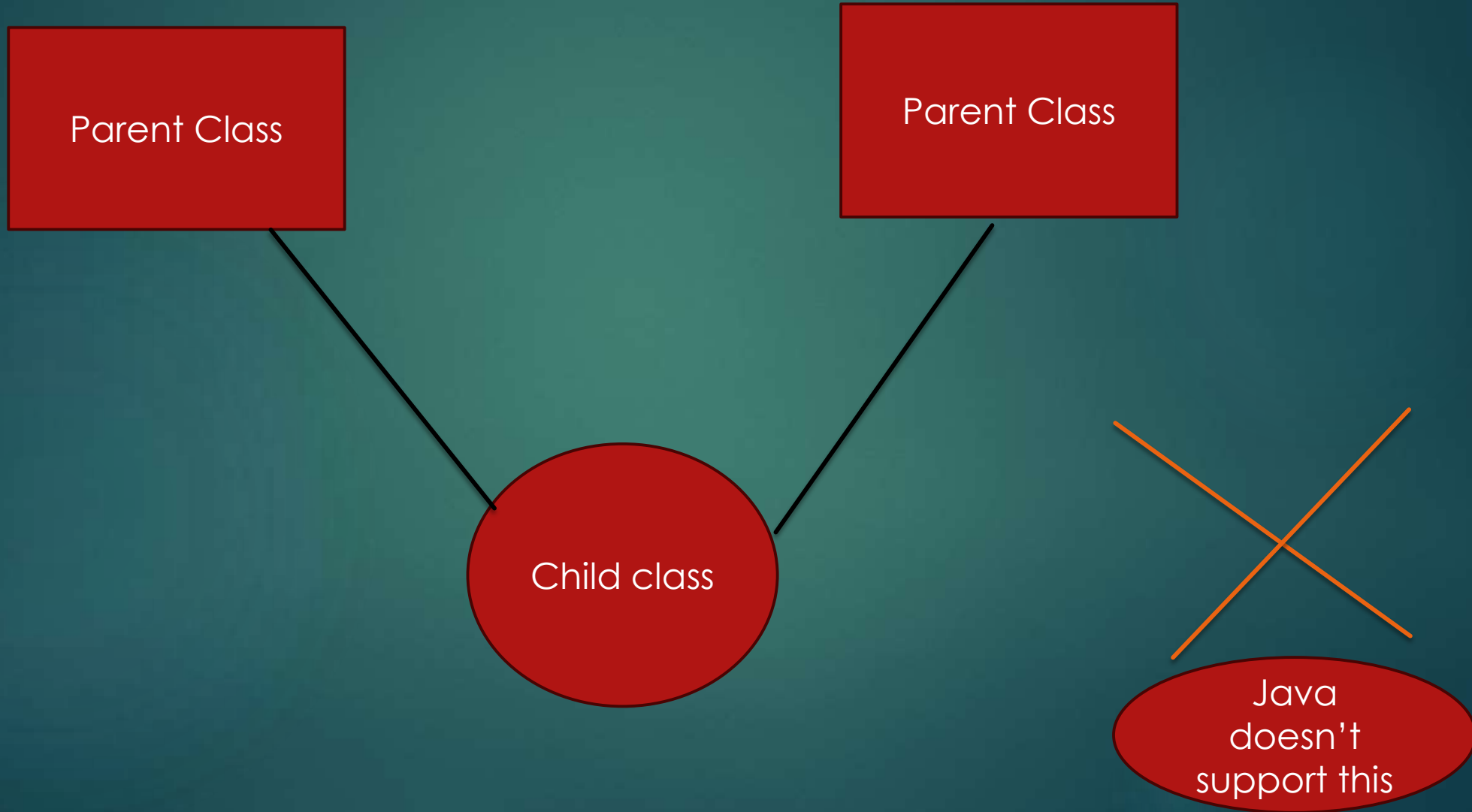
Multi-Level Inheritance



Hierarchical Inheritance



Multiple Inheritance



Software Engineering

- ▶ Software engineering is a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software. It involves the application of engineering principles to software development to ensure that software is reliable, efficient, maintainable, and scalable.

Key Concepts in Software Engineering

- **Software Development Life Cycle (SDLC):**
- **Requirement Analysis:** Understanding and documenting what is required by the user and stakeholders.
- **System Design:** Creating the architecture and design of the software system.
- **Implementation (Coding):** Writing the actual code according to the design specifications.
- **Testing:** Verifying and validating that the software meets the requirements and is free of defects.
- **Deployment:** Releasing the software to the user environment.
- **Maintenance:** Updating and improving the software over time.

SDLC-Software Development Life Cycle

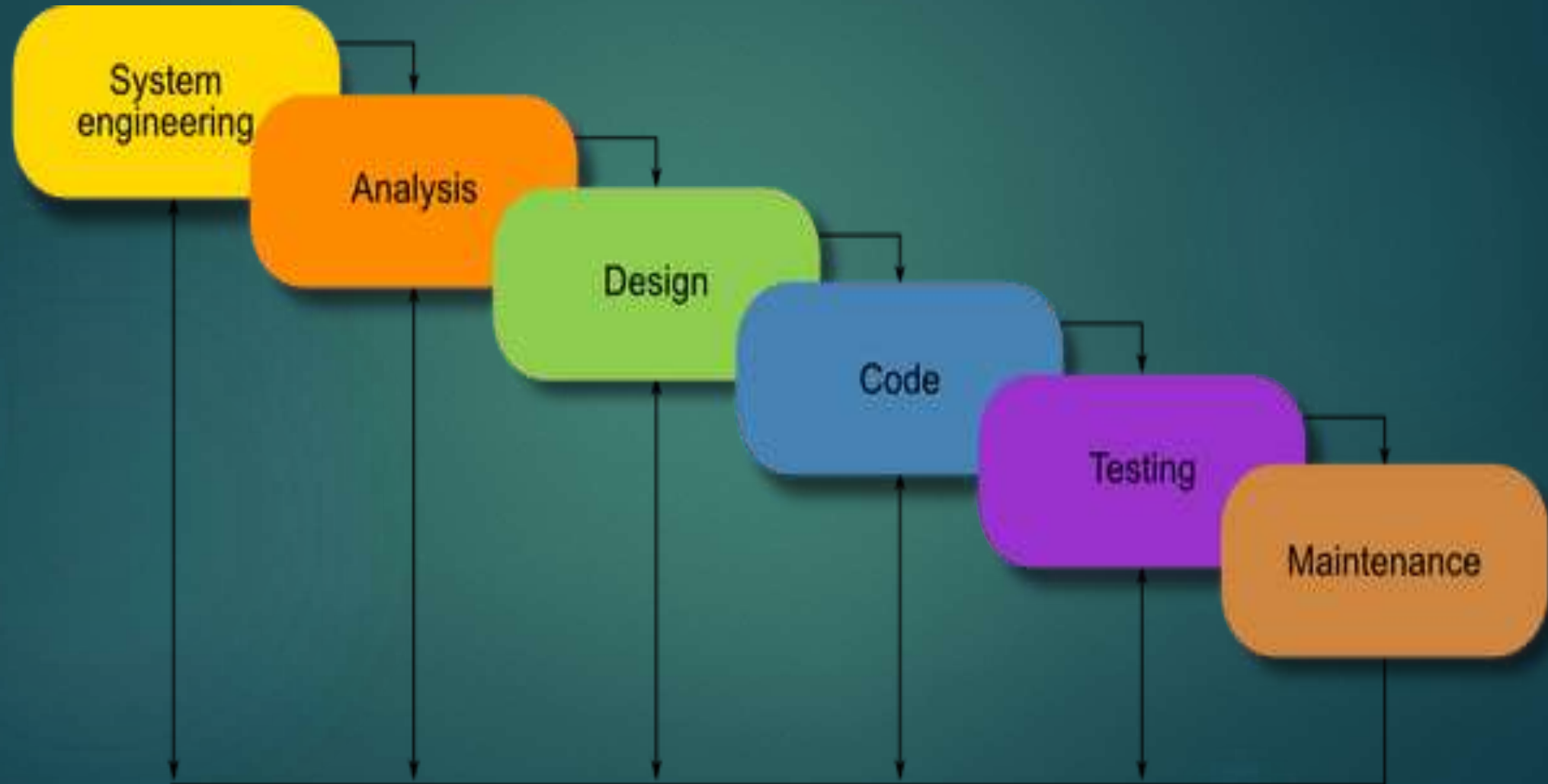
- ▶ This describes the sequence of phases or steps to develop any software.
- ▶ SDLC contains three main stages:
 1. Conception stage
 2. Implementation stage - Coding
 3. Maintenance – Upgrade

SDLC



SDLC Model

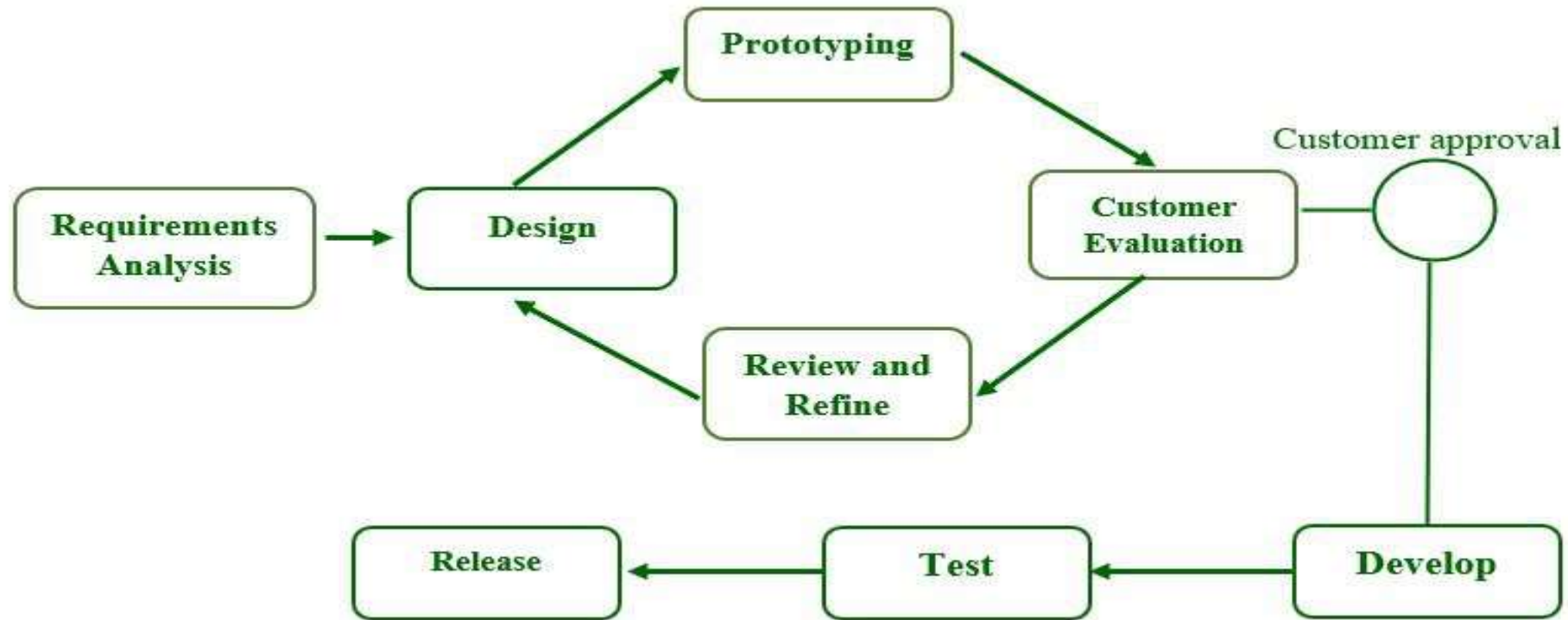
- ▶ The SDLC Model is classified into three categories based on there advantages and disadvantages.
- ▶ Waterfall Model (Classical Model)
- ▶ Prototype Model
- ▶ Spiral Model



Why we call this model as Water Fall

- ▶ The **Waterfall model** is one of the earliest methodologies used in software development and systems engineering. It is a linear and sequential approach where each phase of the development process flows into the next, much like a waterfall.

Prototype Model



Prototype Model

- ▶ The Prototype Model is a systems development methodology in software engineering where a prototype (an early approximation of a final system or product) is built, tested, and then reworked until an acceptable prototype is achieved. This model is particularly useful for projects where the requirements are not well understood from the beginning.

Here are the steps involved in the Prototype Model:

► Requirements Gathering and Analysis

- **Objective:** Understand the basic requirements of the system.
- **Activities:**
 - Gather initial requirements through discussions with stakeholders.
 - Identify the primary functionalities that the system should have.
 - Determine what aspects of the system are uncertain or likely to change.

► 2. Quick Design

- **Objective:** Create a basic design of the system focusing on the aspects that are not well understood.
- **Activities:**
 - Develop a simplified and preliminary version of the system architecture.
 - Focus on user interfaces, input, and output processes.
 - Create mock-ups or sketches of the system.

Con...

► Building the Prototype

- **Objective:** Develop an initial version of the system (the prototype) based on the quick design.
- **Activities:**
 - Implement the basic functionalities of the system.
 - Ensure the prototype includes enough features to demonstrate the core concepts.
 - Use tools and techniques that allow rapid development and iteration.

► 4. User Evaluation

- **Objective:** Gather feedback from users on the prototype.
- **Activities:**
 - Present the prototype to stakeholders and end-users.
 - Collect feedback on the prototype's functionality, design, and usability.
 - Identify any gaps or misunderstandings in the initial requirements.

Con..

► Refining the Prototype

- **Objective:** Improve the prototype based on user feedback.
- **Activities:**
 - Analyze the feedback and determine necessary changes.
 - Modify the prototype to incorporate feedback and new requirements.
 - Iterate this process until the prototype meets user expectations and requirements.

► 6. Engineer the Product

- **Objective:** Transition from the prototype to the final product.
- **Activities:**
 - Finalize the system requirements based on the refined prototype.
 - Design the complete system architecture.
 - Develop the final system using standard software development practices.
 - Perform thorough testing to ensure the system meets all requirements and quality standards.

Cont....

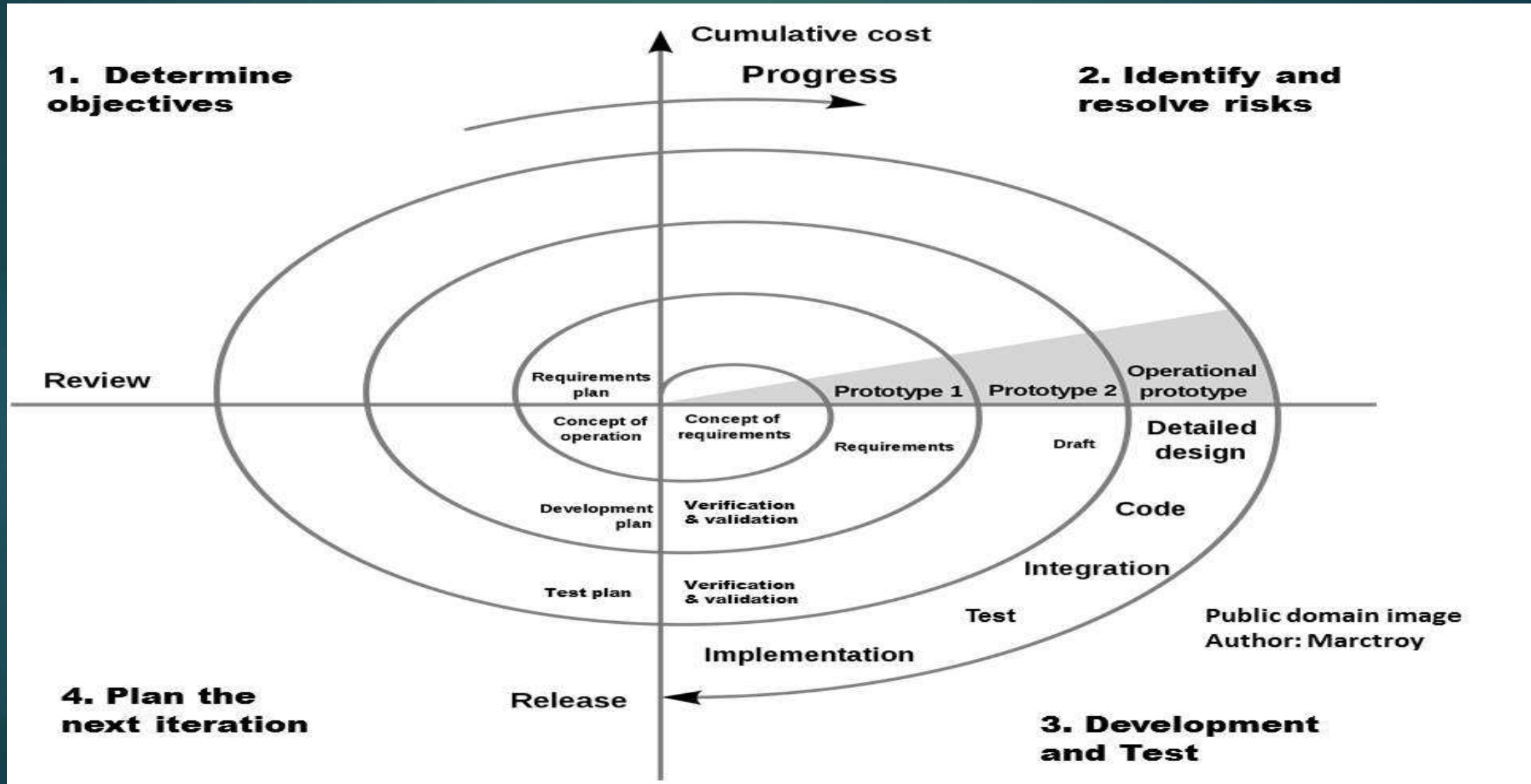
► **System Testing and Implementation**

- **Objective:** Ensure the system is fully functional and ready for deployment.
- **Activities:**
 - Conduct detailed system testing including unit testing, integration testing, and system testing.
 - Validate the system against user requirements.
 - Implement the system in the production environment.
 - Provide training and support to users.

► **8. Maintenance and Updates**

- **Objective:** Maintain the system and implement updates as necessary.
- **Activities:**
 - Monitor the system for any issues or bugs.
 - Perform regular maintenance and updates based on user feedback and changing requirements.
 - Ensure the system remains functional and relevant over time.

Spiral Model



Con....

- ▶ The Spiral Model is a risk-driven software development process model that combines elements of both design and prototyping-in-stages. It is intended to be used for large, complex, and high-risk projects. The model was first introduced by Barry Boehm in 1986 and emphasizes iterative development, allowing for continuous refinement and validation of the software throughout its lifecycle.

Key Phases of the Spiral Model

- ▶ The Spiral Model consists of several iterations or "spirals," each representing a phase of the software development process. Each spiral has four main phases:

1. **Planning:**

1. This phase involves gathering requirements and understanding the objectives of the project. It includes identifying stakeholders, defining requirements, setting constraints, and assessing risks.

2. **Risk Analysis:**

1. In this phase, potential risks are identified, analyzed, and prioritized. This is a critical aspect of the Spiral Model, as it focuses on mitigating risks early in the development process. The outcome of this phase often includes a detailed risk management plan.

3. **Engineering:**

1. This phase involves the actual development of the software, including design, coding, testing, and validation. The engineering phase may include building prototypes to better understand the requirements and test solutions.

4. **Evaluation:**

1. After each iteration, the product and process are evaluated. This includes assessing the deliverables, ensuring that they meet the requirements, and evaluating the overall progress and direction of the project. Stakeholders provide feedback, which is used to improve subsequent iterations.